Scolar: installation guide

White-label e-learning platform (learner app + admin) for a school, academy or coach.

The stack

Layer	Service	Used for
App	Next.js 16	Website, learner app and admin
Hosting	Vercel	Deploys, domain, environment variables
Auth and database	Supabase	Sign-in, two-step sign-in, Postgres with row-level security, scheduled jobs
File storage	Cloudflare R2	Logos, course covers, lesson videos, documents, avatars
Email	Resend	Receipts, course emails, support replies, sign-up and password emails
Payments	Paystack	Course checkout (NGN)
AI (optional)	Cloudflare Workers AI	Study assistant and course writing tools
Web reading (optional)	Firecrawl	Lets the AI course writer read web pages
Code	GitHub	One repo per client

Environment variables

Put these in `.env.local` for local work, and the same values in Vercel > Settings > Environment Variables.

Variable	Required	Where it comes from
NEXT_PUBLIC_SITE_URL	Yes	The live URL, e.g. https://yourschool.com (http://localhost:3010 locally)
NEXT_PUBLIC_SITE_NAME	Yes	Client name; fallback until Branding is saved
NEXT_PUBLIC_SUPABASE_URL	Yes	Supabase > Project Settings > API Keys
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Yes	Supabase > API Keys (sb_publishable_...)
SUPABASE_SECRET_KEY	Yes	Supabase > API Keys (sb_secret_...), server only
SUPABASE_ACCESS_TOKEN	Yes	supabase.com/dashboard/account/tokens; pushes sign-in email designs
R2_ACCOUNT_ID	Yes	Cloudflare > R2 overview

Variable	Required	Where it comes from
R2_BUCKET	Yes	Name of the R2 bucket
R2_ACCESS_KEY_ID	Yes	R2 > Manage API tokens
R2_SECRET_ACCESS_KEY	Yes	R2 > Manage API tokens
R2_ENDPOINT	Yes	https://ACCOUNT-ID.r2.cloudflarestorage.com
NEXT_PUBLIC_R2_PUBLIC_URL	Yes	Bucket custom domain, e.g. https://files.yourschool.com
RESEND_API_KEY	Yes	Resend > API Keys
EMAIL_FROM	Yes	Sending address on the verified domain, e.g. support@yourschool.com
EMAIL_REPLY_TO	No	Inbox that receives replies
CONTACT_INBOX	No	Contact form inbox if no support email is set in admin
EMAIL_ASSET_URL	No	Base URL for email images; defaults to the site URL
PAYSTACK_SECRET_KEY	Yes	Paystack > Settings > API Keys (test first, then live)
CRON_SECRET	Yes	Any long random string (openssl rand -hex 32); same value in Supabase Vault as cron_secret
CLOUDFLARE_ACCOUNT_ID	No	Cloudflare dashboard; enables AI features
CLOUDFLARE_AI_TOKEN	No	Cloudflare > API Tokens (Workers AI)
FIRECRAWL_API_KEY	No	firecrawl.dev

Used only when running the setup scripts in stage 4: SUPABASE_PROJECT_REF, EMAIL_FROM_ADDRESS, EMAIL_SENDER_NAME, BRAND_NAME, BRAND_COLOR, BRAND_LOGO_URL.

This guide takes a fresh copy of Scolar to a live, branded e-learning site for one client (a school, academy or coach). Plan for about two working days. Day 1 covers stages 0 to 6 (accounts, database, storage, email, payments, deploy). Day 2 covers stages 7 to 10 (branding, pages, content and QA).

Work through the stages in order, because each one uses values from the one before. Write the values into `.env.local` as you go, and copy the same file into Vercel at stage 6.

Stage	What	Time
0	Collect client details	30 min
1	Copy the template	15 min
2	Supabase: database and auth	45 min
3	Cloudflare R2: file storage	30 min
4	Resend: email	45 min (DNS can take longer)
5	Paystack: payments	20 min
6	Vercel: deploy, domain, scheduled jobs	45 min
7	Branding and admin settings	30 min
8	Pages, legal text and help centre	2 to 4 hrs
9	Content: categories, courses, team	Client-led
10	QA checklist and handover	1 to 2 hrs

Stage 0. Collect client details

Before you start, get these from the client:

- School or brand name, a short tagline, and the domain (for example `yourschool.com`).
- Logo files:
  - a PNG logo for light backgrounds;
  - optionally a PNG logo for dark backgrounds;
  - a square icon, at least 512 x 512. Ask for PNG, not WebP: email clients such as Outlook do not show WebP, and the logo is used in emails.
- Brand colour (hex, for example `#7c3aed`) and a preferred font, if they have one. Scolar offers Geist, Inter, DM Sans, Plus Jakarta Sans, Manrope and Poppins.
- A support email address, plus the legal entity name and address for the legal pages.
- Access to their domain's DNS (needed for Resend and the site domain).
- A Paystack business account (they own it, so payouts go to them).
- The refund window and the course access policy (for example "refund within 14 days if under 30% completed").

Decide who owns each account. Ideally the client owns Supabase, Cloudflare, Resend, Paystack and Vercel, and adds you as a member. If you host it for them, note that in the handover.

Stage 1. Copy the template

```
cp -R ~/Desktop/TEMPLATES/scolar ~/Desktop/<client-folder>/<client-app>
cd ~/Desktop/<client-folder>/<client-app>
rm -rf .git .next node_modules && git init
npm install
cp .env.example .env.local
```

- In `package.json`, set `"name"` to the client's app name.
- In `.env.local`, set `NEXT_PUBLIC_SITE_NAME` to the client's name. This is only a fallback; the real brand is set in the admin at stage 7.
- Create the client's GitHub repo and push.

Never edit the template itself for a client. Fix bugs in the template first, then copy.

Stage 2. Supabase: database and auth

1. Create a new project at supabase.com (a region close to the client's users; for Nigeria, `eu-west` works well). Save the database password somewhere safe.
2. **Run the schema.** Open SQL Editor > New query. Paste the whole of `supabase/schema.sql` and run it, then do the same with `supabase/seed.sql` (starter course categories). Both must finish with no errors. The schema creates every table, function, row-level security policy and trigger the app uses, so nothing else needs migrating.
3. **Keys.** Go to Project Settings > API Keys and copy these into `.env.local`:
   - Project URL goes in `NEXT_PUBLIC_SUPABASE_URL`.
   - The publishable key (`sb_publishable_...`) goes in `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`.
   - The secret key (`sb_secret_...`) goes in `SUPABASE_SECRET_KEY`. It is server only; never expose it.

   Then create a personal access token at supabase.com/dashboard/account/tokens and put it in `SUPABASE_ACCESS_TOKEN`. The admin uses it to push sign-in email designs to Supabase.
4. **Auth settings.** Go to Authentication > URL Configuration:
   - Site URL: `https://yourschool.com` (use `http://localhost:3010` until the domain is live, then change it).
   - Redirect URLs: add `https://yourschool.com/**` and `http://localhost:3010/**`.
5. Go to Authentication > Sign In / Providers > Email. Keep "Confirm email" on.

6. Optional: under Authentication > Multi-Factor, keep TOTP enabled. Learners and staff can then turn on two-step sign-in in Settings > Security.

7. **First admin.** Run the app locally (`npm run dev`, then open http://localhost:3010/signup) and sign up with the client's main email. Confirm the email, then run this in the SQL Editor:

   ```
   insert into public.user_roles (user_id, role)
   select id, 'super_admin' from auth.users where email = 'owner@yourschool.com';
   ```

   Sign in at http://localhost:3010/admin/login. From now on, add other staff from the admin (Team).

Roles: `super_admin` (everything), `editor` (content, blog, announcements), `accountant` (financials, coupons), `tutor` (shown as "Instructor": builds courses) and `student` (everyone who signs up).

Optional check before running the schema on Supabase: if you have Postgres installed locally, `bash scripts/verify/check-schema.sh` applies the schema and seed to a throwaway database and reports the table count.

Stage 3. Cloudflare R2: file storage

Every upload goes to R2: course covers, lesson videos, documents, avatars, blog images and brand logos. Supabase is used for auth and the database only.

1. In the Cloudflare dashboard, go to R2 and create a bucket (for example `yourschool-files`). Put its name in `R2_BUCKET`.

2. **Public access.** Under the bucket's Settings > Custom Domains, connect `files.yourschool.com` (the domain must be on Cloudflare). Put `https://files.yourschool.com` in `NEXT_PUBLIC_R2_PUBLIC_URL`. While testing, the `r2.dev` public URL also works.

3. **API token.** Go to R2 > Manage API tokens > Create token, with "Object Read & Write" scoped to this bucket. Copy:

   - Access Key ID into `R2_ACCESS_KEY_ID`.
   - Secret Access Key into `R2_SECRET_ACCESS_KEY`.
   - The account ID (shown on the R2 overview page) into `R2_ACCOUNT_ID`.
   - `https://<account-id>.r2.cloudflarestorage.com` into `R2_ENDPOINT`.

4. **CORS.** Browsers upload straight to R2 using signed URLs, so the bucket must allow the site. Go to bucket Settings > CORS policy and paste:

```
[
  {
    "AllowedOrigins": ["https://yourschool.com", "http://localhost:3010"],
    "AllowedMethods": ["GET", "PUT", "HEAD"],
    "AllowedHeaders": ["*"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3600
  }
]
```

Without this, uploads fail with a CORS error in the browser console.

Stage 4. Resend: email

The app sends receipts, enrolment and certificate emails, support replies and announcements through Resend. Supabase sign-up and password emails also go through Resend (step 4 below).

1. In Resend, go to Domains > Add domain and add `yourschool.com` (or a subdomain such as `mail.yourschool.com`). Add the DNS records it shows, then wait for "Verified".
2. Go to API Keys > Create, with "Sending access". Put the key in `RESEND_API_KEY`.
3. Set `EMAIL_FROM=support@yourschool.com`, which must be on the verified domain. The display name ("Your School") comes from the brand settings at stage 7, so set only the address here. Set `EMAIL_REPLY_TO` if replies should go to a different inbox.
4. **Point Supabase Auth at Resend** (custom SMTP plus branded templates). Run this once:

```
# 1. Build the sign-in email templates with the client's brand
BRAND_NAME="Your School" BRAND_COLOR="#7c3aed" BRAND_LOGO_URL="https://files.yourschool.com/brand/logo.png" \
  npx tsx scripts/build-auth-templates.ts

# 2. Install SMTP settings and the templates on the Supabase project
SUPABASE_PROJECT_REF=<ref from the Supabase URL> SUPABASE_ACCESS_TOKEN=sbp_... RESEND_API_KEY=re_... \
EMAIL_FROM_ADDRESS=support@yourschool.com EMAIL_SENDER_NAME="Your School" \
  node scripts/configure-auth-email.mjs
```

If the logo is not uploaded yet, skip `BRAND_LOGO_URL`. After branding at stage 7, use **Admin > Emails > Update sign-in emails in Supabase**, which rebuilds and pushes the templates with the saved brand.

Email templates use tokens that are filled from the brand when emails are sent. Use them when you edit templates in Admin > Emails:

- `{{brand}}`: the brand name.
- `{{brand_color}}`: the accent colour.
- `{{brand_header}}`: the logo, or the name if there is no logo.

The banner images in `public/email/` are generic but use the default blue. If the client's colour is very different, replace them with PNGs of the same names and size (1200 x 440).

Stage 5. Paystack: payments

1. In the Paystack dashboard, go to Settings > API Keys & Webhooks.
2. Put the secret key in `PAYSTACK_SECRET_KEY`. Use `sk_test_...` until QA is done, then switch to `sk_live_...`.
3. Set the webhook URL to `https://yourschool.com/api/paystack/webhook`. Payments are confirmed by the webhook (and by the callback at `/api/paystack/callback` when the buyer returns), so the webhook must be set before going live.
4. Prices are stored in kobo with currency NGN by default. Set the price for each course in the course editor.

Free courses need no Paystack setup at all.

Stage 6. Vercel: deploy, domain and scheduled jobs

1. Import the client's GitHub repo in Vercel (framework: Next.js). `vercel.json` sets the region to `dub1`; change it if the Supabase project is elsewhere, and keep the two close.
2. Add every variable from `.env.local` to Settings > Environment Variables for Production (and Preview if used). Set `NEXT_PUBLIC_SITE_URL=https://yourschool.com` in production.
3. Make up a long random string for `CRON_SECRET` (for example from `openssl rand -hex 32`) and add it too.
4. Deploy, then add the domain under Settings > Domains and set up DNS as Vercel describes.
5. Back in Supabase (stage 2, step 4), change the Site URL to the live domain.
6. **Scheduled jobs** (housekeeping plus abandoned-checkout reminder emails):
    - In Supabase, go to Database > Extensions and enable `pg_cron` and `pg_net`.
    - Go to Project Settings > Vault > Add secret, with name `cron_secret` and the same value as `CRON_SECRET`.
    - Open `supabase/cron.sql`, replace `https://YOUR-DOMAIN` with the live URL, and run it in the SQL Editor.

Stage 7. Branding and admin settings

Sign in at `https://yourschool.com/admin/login` as the super admin.

1. **Settings > Branding.** Set:

- name, sender name (the "from" name on emails) and tagline;
- logo, dark-mode logo and square icon (the uploads go to R2);
- accent colour and the text colour shown on it (the swatches help; keep text contrast readable);
- font, and the default theme (light or dark).

The preview updates live. Saving updates the whole site, the browser tab icon, the install-as-app manifest, share images and every email.

2. **Admin > Emails > Update sign-in emails in Supabase.** Run this after every brand change, so sign-up and password emails match.
3. **Settings > Platform.** Set the support email (shown on the contact page and legal pages, and where contact form messages go) and the other platform details.
4. **Settings > Learning.** Set the name and title of the person who signs certificates.
5. **Settings > Features.** Turn the AI tools off, trial them with admins only, or launch them to everyone. They need `CLOUDFLARE_ACCOUNT_ID` and `CLOUDFLARE_AI_TOKEN` (Workers AI token from the Cloudflare dashboard). Without them the AI features stay hidden.
6. **Settings > Appearance.** Set the background of the welcome banner on the learner dashboard.
7. **Settings > Security.** Turn on two-step sign-in for your own admin account.

Stage 8. Pages, legal text and help centre

This is the only stage that should need code edits. The app logic stays as it is; you change copy and layout.

What	File
Home page	`src/app/(public)/page.tsx`
About page	`src/app/(public)/about/page.tsx`
Courses catalogue	`src/app/(public)/courses/page.tsx`
Contact page	`src/app/(public)/contact/page.tsx`
Public header and footer nav	`src/app/(public)/PublicFrame.tsx`
Help centre FAQs and contact topics	`src/lib/help.ts`
Terms, privacy and refund pages	`src/app/(legal)/*/page.tsx`
Onboarding slides (mobile first run)	`public/onboarding/slide-1..3.jpg` and `src/app/welcome/`
App icons (used when no icon is uploaded)	`public/icons/*.png`, `src/app/favicon.ico`
"Early member" badge cut-off date	`EARLY_UNTIL` in `src/lib/badges.ts`

Legal pages:

- Fill in name, domain and contact email automatically from the brand and settings.
- Replace every `[DATE]` and any bracketed placeholder.
- Have the client approve the refund and privacy wording, because it is their legal text.
- Make sure the help centre answers in `src/lib/help.ts` match the client's actual refund window and access period.

Colours in CSS come from the `--brand` variables (`--brand`, `--brand-fg`, `--brand-strong`, `--brand-deep`, `--brand-soft`, `--brand-pale`). When you add styles, use these variables rather than hex values, so the client can still re-brand from the admin. Use solid fills, not translucent tints.

Database changes for a single client go in `supabase/migrations/0001_name.sql` and so on. Run them in the SQL Editor after `schema.sql`, then regenerate types:

```
psql "<connection string>" -tA -f scripts/schema.sql > /tmp/schema.json && python3 scripts/gen-types.py /tmp/schema.json
```

Stage 9. Content

With the client, or handed over to them:

1. **Admin > Courses > Categories.** Rename or delete the 12 starter categories to fit the school.
2. **Admin > Team.** Invite staff and give each a role. Instructors can build courses; editors manage the blog and announcements.
3. **Admin > Courses > New course.** For each course, add the title, cover, level, price, sections, lessons (video, text, documents) and quizzes, then publish.
4. **Admin > Coupons**, for any launch discounts.
5. **Admin > Blog** and **Announcements**, optional.
6. **Admin > Documents**, for shared files such as a handbook or timetable.

Stage 10. QA checklist and handover

Test on the live domain, on a phone and on a desktop, in light and dark mode.

Accounts

- [ ] Sign up with a new email. The confirmation email arrives, is branded, and the link signs you in.
- [ ] Forgot password: the reset email arrives and setting a new password works.
- [ ] Two-step sign-in can be turned on and is asked for at the next sign-in.

Learning

- [ ] Enrol in a free course. It appears in the dashboard, and the lessons, progress and quizzes work.
- [ ] Finish a course. The certificate appears, the email arrives, and `/certificate` verifies its ID.

Payments (test keys)

- [ ] Buy a paid course with a Paystack test card. Access is granted, a receipt email arrives, and the payment shows in Admin > Financials.
- [ ] Apply a coupon at checkout. The discount shows, and Financials records it.
- [ ] Start a checkout and abandon it. A reminder email arrives later (this needs stage 6 cron).

Admin and support

- [ ] Upload a course cover and a lesson video. The files load from the R2 domain.
- [ ] Send a message from Support as a learner, reply from Admin > Chats, and check the learner sees it with a notification.
- [ ] Use the contact form. The message reaches the support email.
- [ ] Change the accent colour in Branding. The site, the tab icon colour and a test email all follow.

Site

- [ ] On a phone, install to the home screen. The name and icon are the client's.
- [ ] `/sitemap.xml` and `/robots.txt` load, and a shared link shows the branded preview image.
- [ ] No "Scolar" or placeholder text is left anywhere (search the source for `[DATE]`, `yourschool` and `Scolar`).

Go live

- [ ] Switch `PAYSTACK_SECRET_KEY` to the live key and redeploy.

Handover notes for the client:

- Admin URL.
- How to add staff.
- Where branding lives.
- That the Supabase, R2, Resend and Paystack accounts are theirs (or that you host them).
- The support contact.